INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Project Report

Advance Visual Programming

AI Text Summerizer _dotNET

Submitted to:

SIR UZAIR HASSAN

Submitted by:

FATIMA

BSCS-VI-A

2322010-40

Department of Computer Science

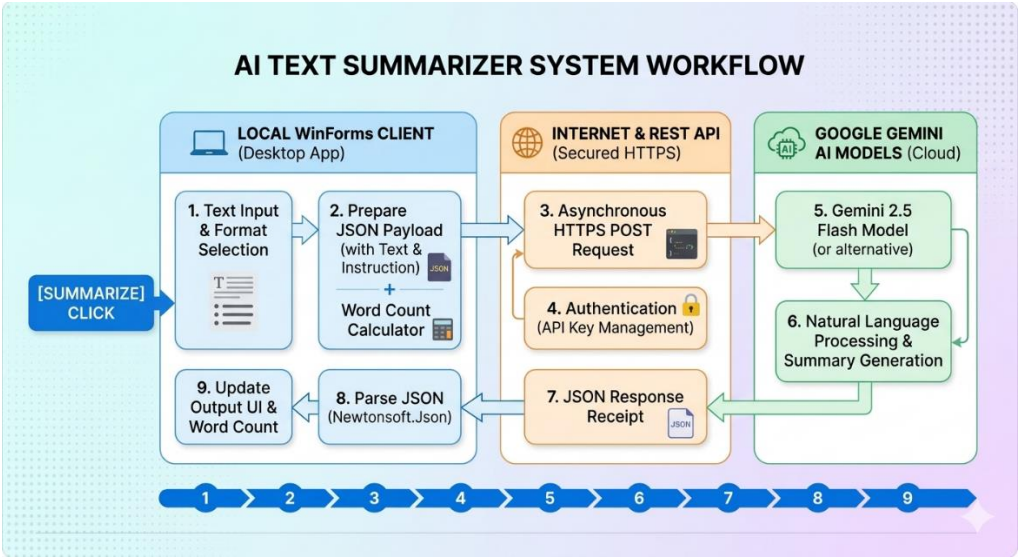
1. Project Introduction

- **Problem Statement:** In an era of massive information overload, reading through lengthy articles, research papers, and reports is highly time-consuming. There is a distinct need for an accessible, offline-initiated desktop tool that can reliably take massive blocks of text and boil them down into clear, structured takeaways without forcing the user to navigate complex web applications.

- **Objective:** The primary objective of this project was to develop a functional desktop application using C# and Windows Forms that communicates asynchronously with advanced large language models (specifically Google Gemini). The system aims to provide users with a clean, side-by-side text summarization interface that supports toggling output structures (Paragraph or Bullet Points) based on user preference.

2. System Design

- **Architecture Diagram:**



- **Workflow:**

  1. The user inputs raw text into the left-side text field and selects a target summary format (Paragraph or Bullet Points).

2. The application captures the input, calculates word counts, and constructs a JSON payload containing the text and specific prompt instructions.
3. The HttpClient transmits an asynchronous POST request containing the payload and user API key to the Google AI Studio endpoint.
4. The model processes the text and returns a JSON response.
5. The application parses the JSON response using Newtonsoft.Json, isolates the isolated text output, and populates it into the read-only output box.

3. Tools & Technologies

- **IDE:** Visual Studio 2022 / 2026 Developer PowerShell.
- **Language & Framework:** C# operating on the .NET Framework (compatible with .NET Core/.NET 6+ implementations).
- **Libraries Used:**
  - Newtonsoft.Json: Utilized heavily for serializing network payloads and deserializing complex, nested JSON objects returned by Google.
  - System.Net.Http: For handling the non-blocking asynchronous POST requests.
- **APIs Leveraged: Google Gemini 2.5 Flash API:** Served as the core generation engine.

4. Code Snippets

Here are the most critical implementations from Form1.cs that handle the core logic:

A. Dynamic Prompt Engineering based on UI Toggle

This snippet controls the instruction passed to the AI depending on which visual button the user clicked.

```
// 3. We change the instruction sent to Gemini based on the button selected!
string promptInstruction = "Summarize the following text concisely in a short paragraph: ";
if (summaryFormat == "Bullet Points")
{
    promptInstruction = "Summarize the following text using clear, concise bullet points (use '-' for bullets): ";
}
```

- **Explanation:** By injecting natural language conditions before the actual raw text, the application forces the deterministic AI model to act as a zero-shot formatter.

B. Asynchronous API Communication

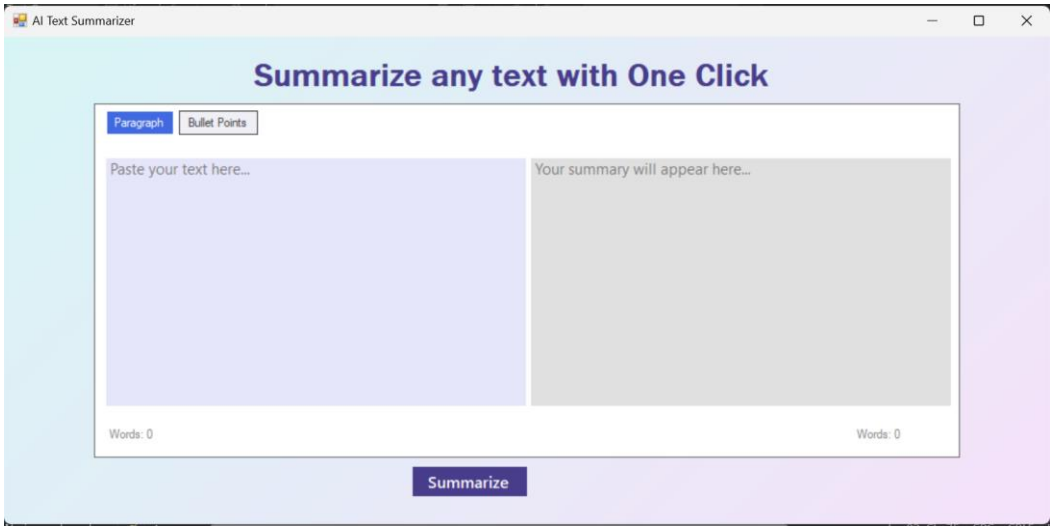
```
// Send the network request
HttpResponseMessage response = await client.PostAsync(url, content);
string responseString = await response.Content.ReadAsStringAsync();

if (response.IsSuccessStatusCode)
{
    JObject json = JObject.Parse(responseString);
    string summary = json["candidates"][0]["content"]["parts"][0]["text"].ToString();
    return summary.Trim();
}
```

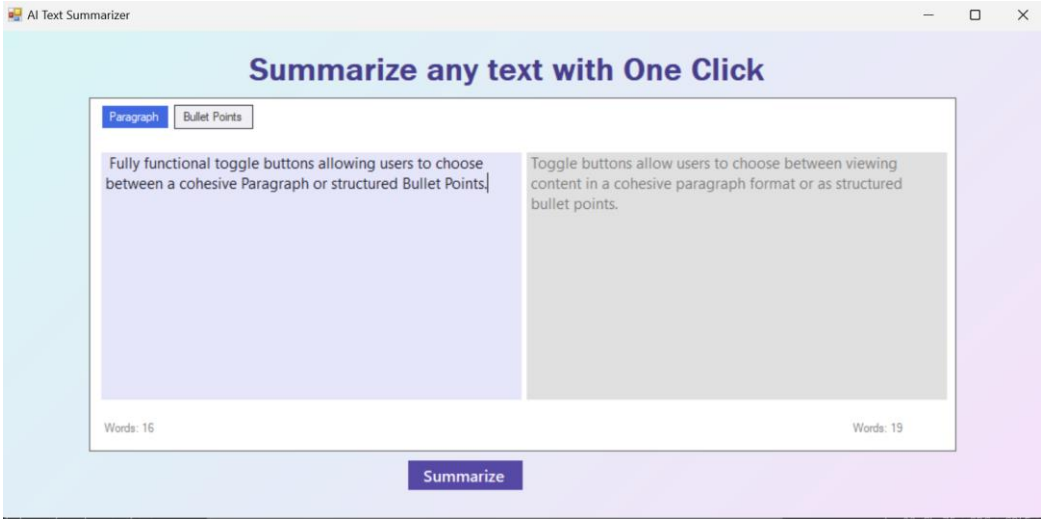
- **Explanation:** Using the await keyword ensures the program UI doesn't "freeze" or stop responding while waiting for the web response. It checks if the code is successful (Status 200) and dives deep into the dynamic JSON map to pull out the raw text.

5. Application Preview

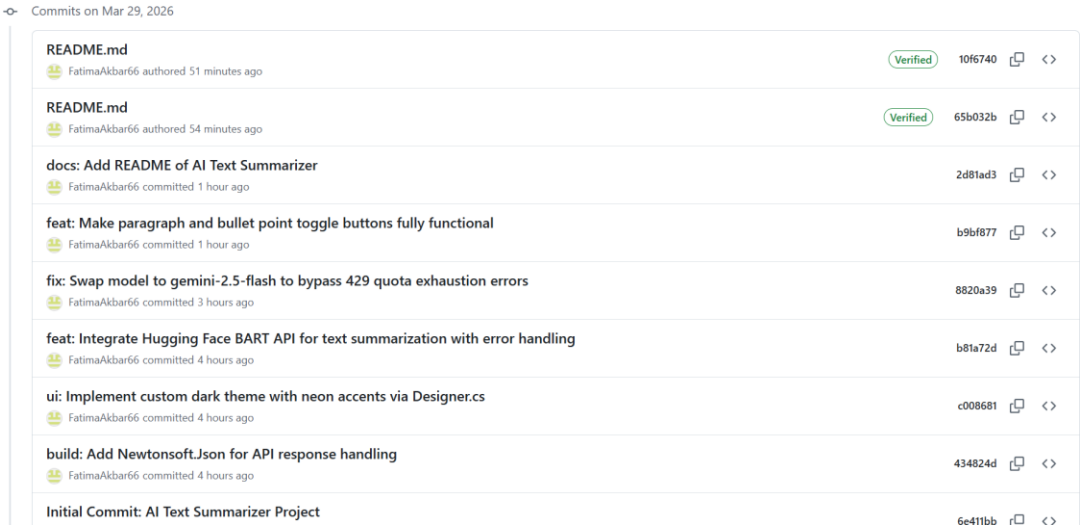
- **UI Layout:**



- **Output:**



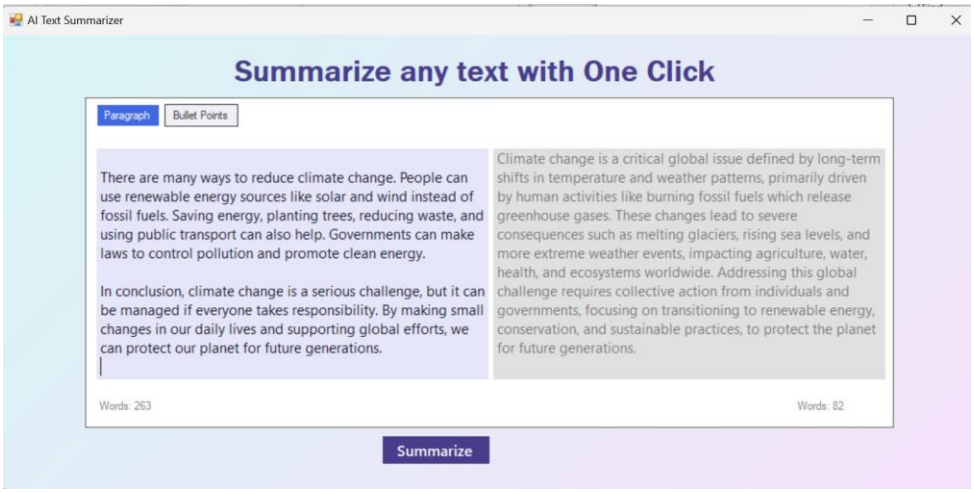
- **Working Process (Git History):**



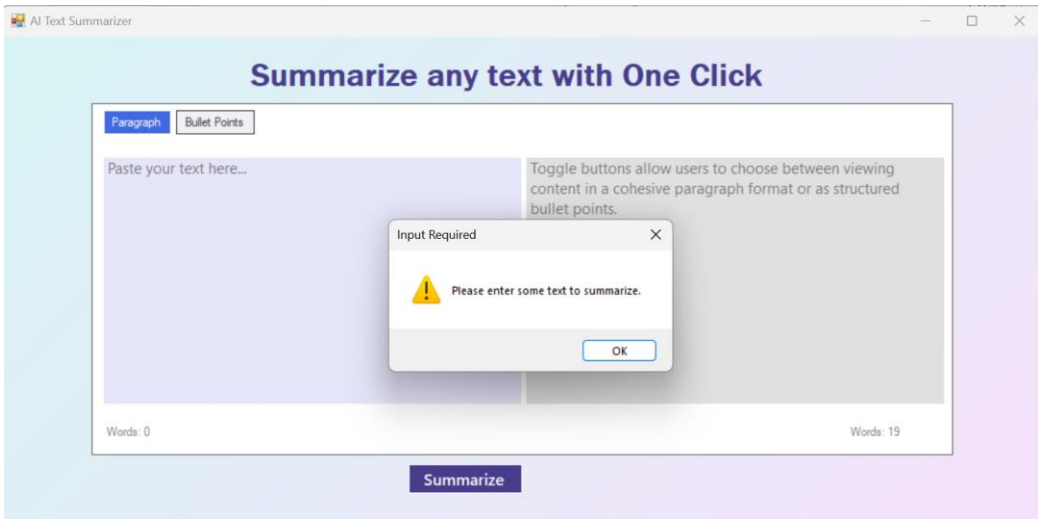
6. Testing

To verify execution and proper error handling, the software was subjected to extreme handling routines:

- **Sample Input 1 (Standard):** Paste a 300-word essay on climate change.
  - **Result Output:** Returns a highly clean 3-sentence summary (or 4 crisp bullet points based on the toggle).



- **Sample Input 2 (Edge Case):** Clicking summarize on empty fields.
  - **Result Output:** A customized MessageBox pop-up prompts the user that input is required, preventing empty credits from being wasted on network requests.



- **Sample Input 3 (Rate Limit Test):** Firing excessive rapid calls.
  - **Result Output:** The system successfully throws custom API Error: 429 catch screens without crashing the local process.

7. Results & Discussion

- **What Worked Well:** The execution of UI elements without locking threads via async/await was smooth.
    - The side-by-side design inspired by modern web apps significantly improved UX readability compared to standard vertical stacks.
    - Switching from the restricted gemini-2.0-flash to the stable gemini-2.5-flash environment bypassed zero-allocation quota locks perfectly.
- **Limitations:**
    - The API key is currently hardcoded into the source strings. This represents a minor security risk for public repositories.
    - There is no local caching; if you summarize the same text twice, it fires another web request and wastes network bandwidth.

8. Future Improvements

1. **Environment Variables:** Relocate API string keys into a git-ignored .env file to practice perfectly secure production coding.
2. **Export Options:** Add a single-click button to automatically save output summaries straight to local .txt or .pdf files.
3. **Local History:** Implement a small SQLite side-database to save previous queries so users can look up past summaries without calling the internet again.